

1. Deployment problem is insufficient memory combined with expensive weight loading.

combined with expensive weight loading.

- model fits memory only if

$$M_{\text{weights}} + M_{\text{KV}} + M_{\text{working}} \leq M_{\text{DRAM}}$$

↑
16 GB

FP16 Model

↑
8 6 3

Jetson Orion Nano.

- weight offloading

backbone weights remain in flash

⇒ loaded when needed.

Note. Keep vision encoder

& KV Cache in memory (LM head)

- on a unified memory system

CPU memory and GPU memory share

the same DRAM pool.

- objective : minimize $\underline{T_{inference}}$ subject to $Accuracy \geq a_{min}$
 $= \text{latency}$

- $\alpha_{\min}$ allow modest accuracy loss.

ⓧ timelines streaming video.

2. Streaming consists of prefill & frame appending & decoding.

VLM inference

write the model as f_{vision} , f_{proj} , f_{LLM}

1. prefill : prompt

$$p = (t_1, \dots, t_m) \rightsquigarrow f_{\text{LLM}}(p) = \left(\begin{array}{c} \text{next-token} \\ \text{prediction, } (k_0) \end{array} \right)$$

discard. init KV cache.

2. frame : frame $\rightsquigarrow$ visual tokens
appending F_i

$$Z_i = f_{\text{proj}} \circ f_{\text{vision}}(F_i)$$

$$\in \mathbb{R}^{B_i \times d_{\text{model}}}$$

$$\rightsquigarrow f_{\text{LLM}}(Z_i; k_{i-1}) = \left(\begin{array}{c} \text{next-token} \\ \text{prediction, } (k_i) \end{array} \right)$$

discard. update KV cache.

$$k_{\text{cache}} \leftarrow [k_{\text{cache}}; k_{\text{new}}]$$

after n frames the cache represents

$$v_{\text{cache}} \leftarrow [v_{\text{cache}}; v_{\text{new}}]$$

$m + (B_1 + \dots + B_n)$ processed tokens.

3. Decoding begins after { explicit query
designated control token

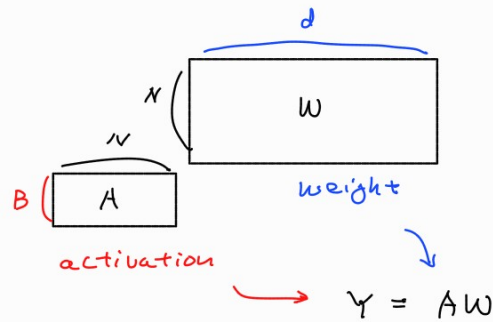
$\curvearrowright$ <RESPOND>

proceed autoregressively : $f_{\text{LLM}}(y_j; k_j^{\text{dec}}) = (y_{j+1}; k_{j+1}^{\text{dec}})$

3. Activation sparsification

selects which input channels contribute to a matrix multiplication.

For one projection

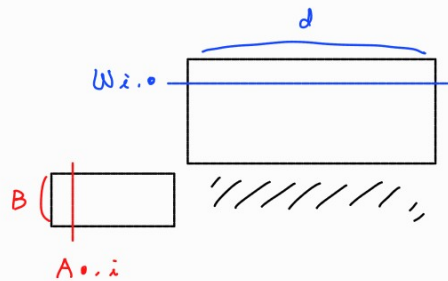


$B = \# \text{ of tokens}$

$N = \# \text{ of input channels}$

$W_{i,\cdot} = \text{weight row associated with channel } i$

output decomposes into channel contributions



$$Y = \sum_{i=0}^{N-1} A_{:,i} W_{i,\cdot}$$

binary mask

$$\Rightarrow \tilde{Y} = A \text{diag}(M) W$$

$$M \in \{0, 1\}^N$$

$$= \sum_i M_i \underbrace{A_{:,i} W_{i,\cdot}}_{\text{only from selected channels}}$$

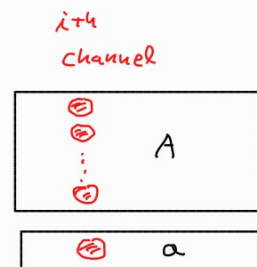
"only rows with $M_i = 1$ need to be loaded"

channel importance

$$v_i = \frac{1}{B} \sum_{b=1}^B |A_{b,i}|$$

= mean

absolute activation



for a single token $v_i = |a_i|$

How to select M ?

channel budget

$$R = \lfloor (1-p) N \rfloor$$

target sparsity

top-R sparsification

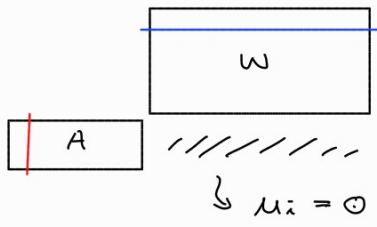
$$\max_{\|M\|_1 \leq R} \sum_i v_i M_i =: I(M)$$

Importance is a proxy for output quality.

t^{th} row $\left\{ \begin{aligned} Y - \tilde{Y} &= \sum_i (1 - \mu_i) A_{:,i} W_{i,:} \\ Y_{t,:} - \tilde{Y}_{t,:} &= \sum_i (1 - \mu_i) A_{t,i} W_{i,:} \end{aligned} \right.$

triangular inequality $\left\{ \begin{aligned} \|Y_{t,:} - \tilde{Y}_{t,:}\|_2 &\leq \sum_i (1 - \mu_i) |A_{t,i}| \|W_{i,:}\|_2 \end{aligned} \right.$

expectation $\left\{ \begin{aligned} \frac{1}{B} \sum_{t=1}^B \|Y_{t,:} - \tilde{Y}_{t,:}\|_2 &\leq \sum_i (1 - \mu_i) \underbrace{v_i}_{\text{drop small-}v_i} \underbrace{\|W_{i,:}\|_2}_{\text{if comparable channels}} \end{aligned} \right.$



- activation magnitude

does not determine downstream task accuracy.

- selection uses $\Rightarrow$ must be repeated
current input's **activation** during inference.

- requires no
activation prediction.

4. Modern VLM importance distribution

are relatively smooth, making storage-aware compromises practical

- $\text{ReLU}(x) = \max(0, x)$
- $\text{SiLU}(x) = \frac{x}{1 + e^{-x}}$ smoother
- $\text{GELU}(x) = x \Phi(x)$
standard normal cumulative distribution function

Example. SwiGLU-style MLP

$$H = \text{SiLU}(X W_{\text{gate}}) \odot (X W_{\text{up}})$$

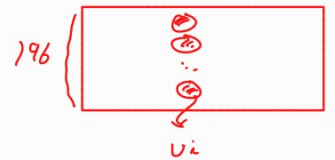
$$Y = H W_{\text{down}}$$

← elementwise multiplication.

⇒ lacks ReLU's $\odot$ 0's

VLM channel importance is also averaged over $\Rightarrow$ visual tokens

$$B = 14 \times 14 = 196.$$



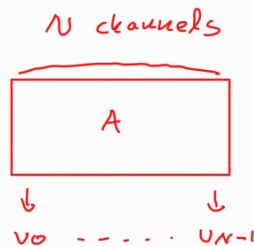
- averaging $\Rightarrow$ token-specific fluctuations reduces

- but it does not guarantee smoothness for every possible activation distribution.

the paper measures

variation

across channels:



$$\mu_v = \frac{1}{N} \sum_i v_i$$

$$\sigma_v^2 = \frac{1}{N} \sum_i (v_i - \mu_i)^2$$

$$CV(v) = \sigma_v / \mu_v$$

Table 1: Coefficient of variation (CV) of neuron importance before the down-projection layer across multiple models.

	VLM					LLM
	LLaVA-7B	LLaVA-0.5B	VILA-8B	NVILA-2B	LongVA	OPT-6.7B
CV	1.44	1.31	1.25	1.07	1.20	11.65
	1.25	1.33	1.38	1.32	1.34	8.63
	3.30	3.58	2.48	4.55	3.01	9.19

Across all VLM models, the CV values (1.07–4.55) are dramatically lower than those of the ReLU-based baseline (8.63–11.65), demonstrating that smooth activation distributions are a consistent property of modern VLM architectures. This smoothing effect makes contiguity-aware selection particularly beneficial for VLMs: when importance differences between neurons are small, I/O efficiency becomes the decisive factor for overall performance.

CV became lower

in latest VLM models!

operating regime on paper

vs

LLM in Flash

$$\rho \approx 40 \sim 60\%$$

↓ sparsity

$$\rho > 90\%$$

[V] moderate importance + neighbouring channels

[X] slightly more + scattered channels importance

Note. individual layers can have very different sparsities.

5. Reducing transferred bytes does not necessarily reduce flash latency.

- if each weight occupies b bytes

$$D(M) = b \|M\|_1$$

↓
of selected channels.

- with throughput $\beta_{eff}(M)$ [byte/sec]

$$L_{IO}(M) = \frac{D(M)}{\beta_{eff}(M)}$$

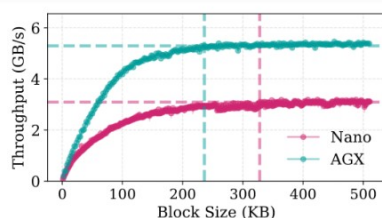
← data [byte]
← throughput [byte/sec]
latency [sec]

Note. flash throughput.

small read $\Rightarrow$ request-processing overhead.

large read $\Rightarrow$ bandwidth limited.

$\beta(\text{request size}) \uparrow \rightarrow \beta_{max}$
contiguous request size. bandwidth ceiling.



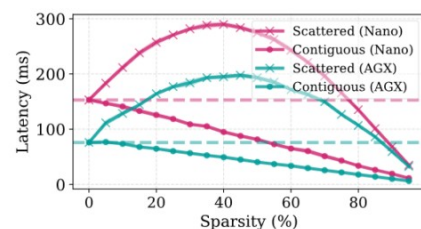
for two access patterns

$$\frac{L_2}{L_1} = \frac{D_2 / D_1}{\beta_2 / \beta_1}$$

data.
throughput.
latency

$$\text{if } \frac{\beta_2}{\beta_1} < \frac{D_2}{D_1}$$

throughput data



$$\text{then } \frac{L_2}{L_1} > 1 \quad \text{even if } \frac{D_2}{D_1} < 1$$

latency data.

- chunking may load more channels at comparable accuracy.

get finish sooner.

$$\begin{aligned} \text{importance } I, \Delta I &\rightarrow \frac{I - \Delta I}{L - \Delta L} > \frac{I}{L} \quad \text{if } \frac{\Delta L}{L} > \frac{\Delta I}{I} \\ \text{latency } L, \Delta L &\rightarrow \end{aligned}$$

large latency reduction small importance loss

6. contiguity distribution summarizes

a mask by the lengths of its consecutive selected runs.

- $C(n)$ = set of all chunks (maximal contiguous runs)

- $h_m(r) = \# \text{ of chunks}$

whose length is r

$$\Rightarrow \|M\|_1 = \sum_r r h_m(r) \rightarrow \# \text{ of selected channels}$$

$$|C(M)| = \sum_r h_M(r) \rightarrow \# \text{ of chunks.}$$

$$\Rightarrow \text{mean chunk length} = \frac{\sum_r r h_n(r)}{\sum_r h_n(r)}$$

chunk $\rightarrow$

keep	discard
length	location
& size	& gaps
	& global arrangement.

- $T[r]$: lookup table of effective per-chunk costs.

- $$\hat{L}(M) = \sum_{c \in C(M)} T[|c|] = \sum_r h_m(r) T[r] \quad (\text{additive approximation})$$

↙

latency
of each chunks

↘

classify
with chunk length.

$$\{1, 2, 4, 6, 7\} = \{1, 2\} \cup \{4\} \cup \{6, 7\}$$

20G weight $\Rightarrow$ need to make
matrices per frame runtime estimation
in LLaVA-OneVision cheap enough

7. The latency table is obtained from saturated, steady-state read experiments.

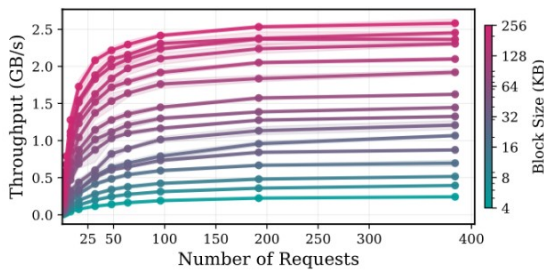
• latency $T[r] \approx \frac{t_{\text{measured}}(Q, r)}{Q} \approx \frac{br}{\beta(br)}$
 $Q \rightarrow$ amortized

Note.

Q in $2^0, 2^1, \dots, 2^9$
 512 1024 2048 ... 5120

- Q equal-sized : large enough chunks to reach stable throughput [byte/sec]
- not latency of isolated workload.

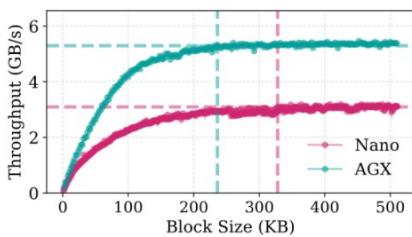
throughput stabilizes



uniform-size

& fixed-size.

& sufficiently many requests.



saturation size

$$z_{\text{sat}} = \min \{ z : \beta(z) \geq 0.99 \beta_{\text{max}} \}$$

$$= \begin{cases} 238 \text{ KB} & \text{AGX} \\ 348 \text{ KB} & \text{Nano} \end{cases}$$

• saturation size should

come from target device's profile

Profiling Setup

1. dummy file (128 MB) in Flash
2. sequential reads of size 1KB, 2KB, ... Δ_{max}
3. record average throughput over Δ_{p} trials

reach 99% of peak throughput.

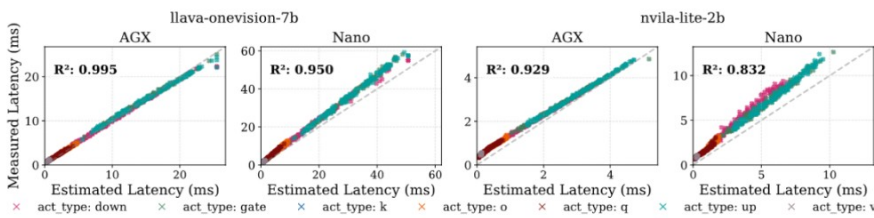
Note. Linux direct I/O

: six-thread C++ pool.

8. The Model is useful because its errors are approximately proportional.

- $L_{\text{measured}} = \alpha \hat{L} + \varepsilon$

- latency bias : uniform profiling $\xrightarrow{\text{reproduce}}$ mixed chunk lengths & strides & controller behavior & queue interactions of real accesses.



linear (high R^2)

weakened averaging effect } smaller model $\rightarrow$ low concurrency
lower-end hardware.
 $\hookrightarrow$ deviations

- $T_{\text{actual}}[c] \approx \alpha T[c]$

- $L_{\text{actual}}(c) \approx \frac{I(c)}{\alpha T[c]} = \frac{1}{\alpha} U(c)$
chunk $\rightarrow$ importance $\rightarrow$ latency

- ranking of chunks unchanged.

- requires

shared multiplicative factor (α)

9. Offline hot-cold reordering increases

the opportunity to form useful contiguous chunks.

Activation Frequency

• calibration inputs $c = 1, 2, \dots, C$

"probability of

i^{th} channel being

top-50% important"

• define $f_i = \frac{1}{C} \sum_{c=1}^C \mathbb{1} [i \in \text{Top50\%}(V_c)]$

channel. input channel importance. input

Hot-Cold Reordering

• sort channels by decreasing f_i

• $W' = P W$ $A' = A P^T$

permutation matrix

• computation preserved

$$A'W' = A P^T P W = A W$$

runtime masks

operate in this reordered coordinate system

$$\tilde{Y} = A' \text{diag}(M') W'$$

reordered.

Reordering yield modest improvement of the original ordering.

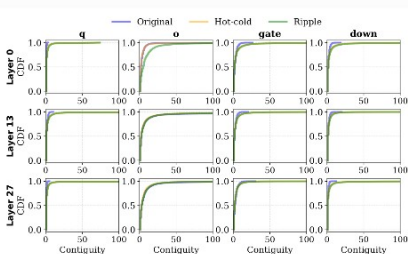


Figure 12: CDF of contiguity of selected neurons before and after reordering, with sparsity=0.4.

• hot-cold. $\Rightarrow$ marginal activation frequencies

• Ripple $\Rightarrow$ coactivation information.

Appendix G

overall difference is minor

• offline ordering $\rightarrow$ best subset for every future input.

• activation-frequency analysis.

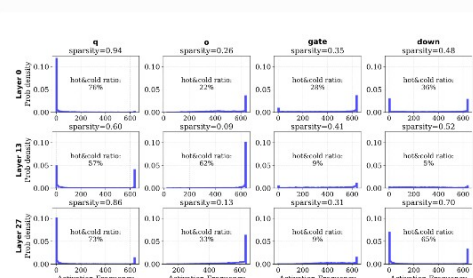


Figure 11: Activation frequency of neurons across layers, with effective sparsity set to 40% (layer-wise sparsity determined by TEAL [24] profiling). The text in the center of each plot indicates the proportion of hot neurons (activated >99% of the time) and cold neurons (<1%).

• frequency g_i $\left\{ \begin{array}{l} > 99\% \text{ Hot} \\ < 1\% \text{ Cold.} \end{array} \right.$

• many channels in between. $\rightarrow$ input dependent selection.

• Layer-specific $\left\{ \begin{array}{l} \text{effective sparsity} : 40\% \\ \text{sparsity (TEAL)} : \text{1st layer \& sparsity} : 94\% \end{array} \right.$

Runtime Permutation operation incurs negligible overhead.

- for one profile down-projection permutation

on Nano : latency = $1.5\text{ms} + \underbrace{0.3\text{ms}}_{95\% \text{ upper confidence.}}$

which is $< 0.02\%$ of total latency

10. The stated selection objective

maximizes retained importance per estimated latency.

$$\max_M \frac{I(M) \rightarrow \text{importance}}{\hat{L}(M) \rightarrow \text{latency}} = \max_M \frac{\sum_i v_i \mu_i}{\sum_i \mu_i T(v_i)}$$

↑ input
mask hardware

(i) $\mu \in \{0, 1\}^N$

(ii) $1 \leq \|M\|_1 \leq R$

candidate : $I(c) = \sum_{i \in c} v_i$
 chunk $U(c) = \frac{I(c) \rightarrow \text{importance}}{T[c]} \rightarrow \text{latency}$

① ranks candidate using $U(c)$
 ② greedily accepts compatible chunks.

Limitation $\frac{I(\mu)}{\hat{L}(\mu)} = \sum_{c \in C(\mu)} \underbrace{\frac{T[c]}{\sum_{c \in C(\mu)} T[c]}}_{w_c} U(c)$ with $\sum_c w_c = 1$

→ weighted average : in theory, it is optimal to pick single chunk with best $U(c)$ within $C(\mu)$

논리적 결론

- 문제 : μ 를 어떻게 고를 것인가?
- 전제 : $R = \lfloor (1-p)N \rfloor$ 은 고정한다.
- 기법 Top-R : $\max_{\|M\|_1=R} I(M)$

• Neuron chunking-1 알고리즘 : $\max_{\|M\|_1=R} \frac{I(\mu)}{\hat{L}(\mu)}$

아하!에 $\|M\|_1 = R$ 이

전제조건이므로 충족하지

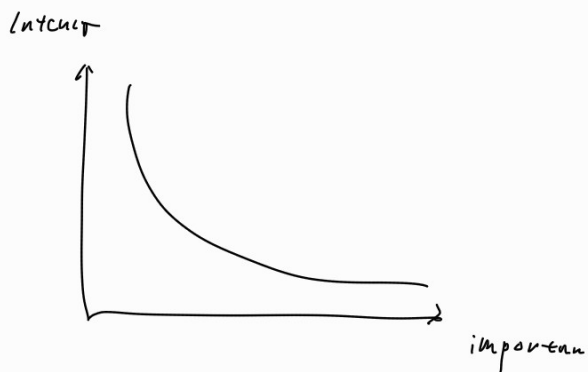
않은 결론이다.

$$\exists \mu \quad I(\mu) = \sum_i v_i \mu_i \geq \text{constant}$$

$$\begin{aligned} \text{minimize} \quad \hat{L}(\mu) &= \sum_i k_{\mu}(v_i) \cdot T[v_i] \\ &= \sum_i (1 + \kappa |c|) \end{aligned}$$

$$\begin{array}{l} \min_{\mu} \quad \underbrace{a K(\mu)}_{\text{channel의 } \frac{1}{T}} + \underbrace{c R(\mu)}_{\text{channel의 } \frac{1}{T}} \quad \text{s.t.} \quad \underbrace{I(\mu)}_{\text{importance의 } \frac{1}{T}} \geq \text{constants.} \end{array}$$

find $M: O(N R_{\max} K_{\max})$



$$I(\mu) = \sum_i v_i \mu_i$$

$$L(\mu) = \sum_c T[|c|]$$

$$= \sum_c a + c |c|$$

$$= a K(\mu) + c R(\mu)$$

논점이 $\frac{1}{T}$ $\frac{1}{T}$ $\frac{1}{T}$

$$R^- \leq R(\mu) \leq R^+$$

$$\text{maximize} \quad \frac{I(\mu)}{a K(\mu) + c R(\mu)}$$

우리가 $\frac{1}{T}$ $\frac{1}{T}$ $\frac{1}{T}$

$$I(\mu) \geq (\text{ratio}) \times (\text{total importance})$$

$$\text{minimize} \quad a K(\mu) + c R(\mu)$$

$$\mu_1 + \mu_2(1-\mu_1) +$$